VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



CO2008 - Computer Architecture Lab

Lab Assignment Report

Class: CC06

Group: 06

Filtering and Prediction Signal with Wiener Filter

Advisor(s):	Nguyen Thien An	
Student(s):	Nguyen Xuan An	2452023
	Pham Gia Hung	2452431
	Dinh Phuc Khang	2452453
	Pham Vu Khoi Nguyen	2412372

HO CHI MINH CITY, DECEMBER 2025

Contents

1 Introduction **3**

2 Methodology **4**

3 System Design and Implementation **5**

3.1 Overall Data Flow 5

3.2 Autocorrelation and Cross-correlation 5

3.3 Solving the Wiener–Hopf Equations 6

3.4 Filter Output and MMSE Computation 6

4 Code Implementation **7**

5 Results and discussions **13**

6 Conclusion **14**

List of Figures

1 Analysis of the Wiener filter on the $N = 10$ sample test 13

Listings

1 Main Wiener filter pipeline (pseudo code) 8

2 Function-level pseudo code for Wiener filter 9

Team Workload

The project was carried out by a team of four students. The workload was distributed as follows:

- **Pham Gia Hung**: Input handling, file I/O, data loading and memory layout for the MIPS program and computation of MMSE.
- **Pham Vu Khoi Nguyen**: Correlation computation module (autocorrelation and cross-correlation) in both reference code and MIPS.
- **Dinh Phuc Khang**: Linear system solver, computation of the filter coefficients, integration and debugging.
- **Nguyen Xuan An**: Report writing, diagram and flowchart preparation, and final integration of the code and documentation.

All members participated in discussions, debugging sessions, and testing of the final program. The responsibilities listed above indicate the primary contributions of each member.

1 Introduction

This report presents the implementation of a Wiener filter for signal filtering and prediction in the context of the *Computer Architecture Lab*. The objective of the assignment is to design and implement a system that can estimate a desired signal from a noisy observation, and to realize the algorithm using MIPS assembly.

The input of the problem consists of discrete-time sequences representing the desired signal and the noisy input signal. Based on these sequences, we design a finite impulse response (FIR) Wiener filter that minimizes the mean squared error (MSE) between the filter output and the desired signal.

In this report, we first review the theoretical background of Wiener filtering, then describe the algorithm and data flow used in our implementation. Afterwards, we present experimental results obtained from a reference implementation in a high-level language and discuss the corresponding MIPS implementation. Finally, we summarize our work and propose possible improvements. [2]



2 Methodology

In the Wiener filtering problem, we consider a discrete-time input signal $x(n)$ which is the sum of a desired signal $s(n)$ and an additive noise $w(n)$:

$$x(n) = s(n) + w(n).$$

The goal is to design a linear filter with impulse response $h(k)$, $0 \leq k \leq M-1$, such that the filter output

$$y(n) = \sum_{k=0}^{M-1} h(k)\, x(n-k)$$

approximates a desired response $d(n)$ in the sense of minimum mean square error (MMSE).

The estimation error is defined as

$$e(n) = d(n) - y(n),$$

and the mean squared error is

$$E_M = \mathbb{E}\{|e(n)|^2\}.$$

By minimizing E_M with respect to the filter coefficients $h(k)$, we obtain the Wiener–Hopf equations:

$$\sum_{k=0}^{M-1} h(k)\, \gamma_{xx}(l-k) = \gamma_{dx}(l), \quad l = 0, 1, \ldots, M-1,$$

where

$$\gamma_{xx}(k) = \mathbb{E}\{x(n)\, x(n-k)\}$$

is the autocorrelation of the input signal, and

$$\gamma_{dx}(k) = \mathbb{E}\{d(n)\, x(n-k)\}$$

is the cross-correlation between the desired signal and the input.

In matrix form, these equations can be written as

$$\mathbf{R}_M\, \mathbf{h}_{\text{opt}} = \boldsymbol{\gamma}_d,$$

where $\mathbf{R}_M$ is an $M \times M$ Toeplitz matrix built from $\gamma_{xx}(k)$, $\boldsymbol{\gamma}_d$ is the cross-correlation



vector, and $\mathbf{h}_{\text{opt}}$ is the optimal filter coefficient vector. The optimal solution is thus

$$\mathbf{h}_{\text{opt}} = \mathbf{R}_M^{-1}\,\boldsymbol{\gamma}_d.$$

The minimum mean squared error achieved by this filter can be expressed as

$$\text{MMSE} = \sigma_d^2 - \boldsymbol{\gamma}_d^T \mathbf{R}_M^{-1}\,\boldsymbol{\gamma}_d,$$

where $\sigma_d^2 = \mathbb{E}\{|d(n)|^2\}$ is the variance of the desired signal.

In our assignment, the statistical expectations are approximated by finite sums over the given input data, and the filter length M is chosen as a small integer (e.g. 3) to simplify the MIPS implementation.

3 System Design and Implementation

3.1 Overall Data Flow

The implementation is divided into several stages:

a) Reading the input sequences from text files.

b) Computing the autocorrelation and cross-correlation values.

c) Forming the Toeplitz autocorrelation matrix and solving the Wiener–Hopf equations to obtain the optimal filter coefficients.

d) Applying the filter to the input sequence to generate the output.

e) Computing the MMSE between the output and the desired sequence.

For verification, we first implemented the algorithm in a high-level language (Python/C++). The verified numerical results are then used as a reference when developing the MIPS assembly program.[1]

3.2 Autocorrelation and Cross-correlation

Given an input sequence $x[n]$ of length N, the autocorrelation values $\gamma_{xx}(k)$ for $k = 0, \ldots, M-1$ are approximated by

$$\gamma_{xx}(k) \approx \frac{1}{N} \sum_{n=k}^{N-1} x[n]\,x[n-k].$$



Similarly, for the desired sequence $d[n]$, the cross-correlation values $\gamma_{dx}(k)$ are approximated by

$$\gamma_{dx}(k) \approx \frac{1}{N} \sum_{n=k}^{N-1} d[n]\, x[n-k].$$

In the MIPS program, these sums are implemented using nested loops over the indices n and k, with intermediate results stored in floating-point registers or in memory, depending on the chosen design.

3.3 Solving the Wiener–Hopf Equations

After computing the correlation values, we construct the Toeplitz matrix $\mathbf{R}_M$ using

$$[\mathbf{R}_M]_{i,j} = \gamma_{xx}(|i-j|), \quad 0 \le i, j \le M-1.$$

The right-hand side vector is

$$\boldsymbol{\gamma}_d = \begin{bmatrix} \gamma_{dx}(0) & \gamma_{dx}(1) & \cdots & \gamma_{dx}(M-1) \end{bmatrix}^T.$$

To obtain the optimal coefficient vector $\mathbf{h}_{\text{opt}}$, we solve the linear system

$$\mathbf{R}_M\, \mathbf{h}_{\text{opt}} = \boldsymbol{\gamma}_d.$$

In our implementation, we use Gaussian elimination with partial pivoting in the high-level reference program. The same algorithm is later translated into MIPS assembly, where each operation (row scaling, row subtraction) corresponds to a small block of arithmetic instructions.

3.4 Filter Output and MMSE Computation

Once the coefficients $h(k)$ are known, the output sequence is computed by the convolution-like operation

$$y(n) = \sum_{k=0}^{M-1} h(k)\, x(n-k),$$

with appropriate boundary handling for indices $n-k<0$.

The estimation error is then

$$e(n) = d(n) - y(n),$$



and the MMSE is computed as

$$\text{MMSE} = \frac{1}{N}\sum_{n=0}^{N-1} e^2(n).$$

In the MIPS program, the output sequence, error sequence, and final MMSE value are stored in predefined memory locations so that they can be printed to the console and written to an output text file as required.

4 Code Implementation

To make the Wiener filter algorithm easier to implement and test, we decompose it into a small set of high-level functions. Each function corresponds to a clear processing step in the overall data flow:

- `readSignalFromFile`: reads the signal samples from a text file.
- `computeAutocorrelation`: computes the autocorrelation $\gamma_{xx}(k)$ of the input signal $x(n)$.
- `computeCrosscorrelation`: computes the cross-correlation $\gamma_{dx}(k)$ between $d(n)$ and $x(n)$.
- `createToeplitzMatrix`: builds the Toeplitz matrix R_M from the values $\gamma_{xx}(k)$.
- `solveLinearSystem`: solves the linear system $R_M \cdot h = \gamma_d$.
- `computeWienerCoefficients`: wraps the steps for computing the Wiener filter coefficients h_{opt}.
- `applyWienerFilter`: applies the filter to the input signal to produce $y(n)$.
- `computeMMSE`: computes the MMSE value from $d(n)$ and $y(n)$.
- `writeOutputToFile`: writes the output sequence and the MMSE value to the file `output.txt`.

The following pseudo-code summarizes the main control flow of the program and shows how these functions are connected.



Listing 1: Main Wiener filter pipeline (pseudo code)

```
main()
    // 1. Read input data
    desiredSignal <- readSignalFromFile("desired.txt", N)
    inputSignal   <- readSignalFromFile("input.txt",   N)

    // 2. Check length constraint
    if length(desiredSignal) != length(inputSignal) then
        print "Error: size not match"
        writeOutputToFileError("output.txt")
        return
    end if

    // 3. Compute correlation values
    autoCorr  <- computeAutocorrelation(inputSignal,  N)
    crossCorr <- computeCrosscorrelation(desiredSignal,
                                         inputSignal, N)

    // 4. Build Wiener-Hopf system
    Rmatrix  <- createToeplitzMatrix(autoCorr, N)
    hOpt     <- computeWienerCoefficients(Rmatrix, crossCorr)
    // (computeWienerCoefficients calls solveLinearSystem)

    // 5. Apply Wiener filter
    outputSignal <- applyWienerFilter(inputSignal, hOpt, N)

    // 6. Compute MMSE
    mmseValue <- computeMMSE(desiredSignal, outputSignal)

    // 7. Write results
    writeOutputToFile(outputSignal, mmseValue, "output.txt")
end main
```

The detailed behavior of each function is described in the following pseudo code.

Listing 2: Function-level pseudo code for Wiener filter

```
function readSignalFromFile(fileName, input, MAX_SIZE)
    open fileName for reading
    count <- 0
    while count < MAX_SIZE and not end of file do
        value <- read next number as float
       input[count] <- value
        count <- count + 1
    end while
    return count                // number of samples read
end function

function computeAutocorrelation(signal, autocorr, N)
    // gamma_xx(k) ~= (1/N) * sum_{n=k}^{N-1} x[n] * x[n-k]
    for k from 0 to N-1 do
        sum <- 0
        for n from k to N-1 do
            sum <- sum +input[n] *input[n - k]
        end for
        autocorr[k] <- sum / N
    end for
end function

function computeCrosscorrelation(desired, input, crosscorr, N
    )
    // gamma_dx(k) ~= (1/N) * sum_{n=k}^{N-1} d[n] * x[n-k]
    for k from 0 to N-1 do
        sum <- 0
        for n from k to N-1 do
            sum <- sum + desired[n] * input[n - k]
        end for
        crosscorr[k] <- sum / N
    end for
end function

```

```
function createToeplitzMatrix(autocorr, R, N)
    // R[i,j] = autocorr[|i - j|]
    for i from 0 to N-1 do
        for j from 0 to N-1 do
            index <- abs(i - j)
            R[i, j] <- autocorr[index]
        end for
    end for
end function

function solveLinearSystem(A, b, x, N)
    // Solve A * x = b using Gauss-Jordan elimination
    for i from 0 to N-1 do
        // find pivot row with largest |A[r,i]|
        pivotRow <- i
        maxVal   <- abs(A[i, i])
        for r from i+1 to N-1 do
            if abs(A[r, i]) > maxVal then
                maxVal   <- abs(A[r, i])
                pivotRow <- r
            end if
        end for

        if maxVal is very small then
            // nearly singular: return zero solution
            for k from 0 to N-1 do
                x[k] <- 0
            end for
            return
        end if

        // swap row i with pivotRow in A and b
        swap rows A[i,*] and A[pivotRow,*]
        swap b[i] and b[pivotRow]

```

```
        // normalize pivot row
        pivot <- A[i, i]
        for c from 0 to N-1 do
            A[i, c] <- A[i, c] / pivot
        end for
        b[i] <- b[i] / pivot
        // eliminate other rows
        for r from 0 to N-1 do
            if r != i then
                factor <- A[r, i]
                for c from 0 to N-1 do
                    A[r, c] <- A[r, c] - factor * A[i, c]
                end for
                b[r] <- b[r] - factor * b[i]
            end if
        end for
    end for
    // after elimination, b contains the solution
    for i from 0 to N-1 do
        x[i] <- b[i]
    end for
end function

function computeWienerCoefficients(desired, input, N,
   coefficients)
    autocorr  <- array of size N
    crosscorr <- array of size N

    computeAutocorrelation(input,  autocorr,  N)
    computeCrosscorrelation(desired, input, crosscorr, N)
    R <- N x N matrix
    createToeplitzMatrix(autocorr, R, N)
    solveLinearSystem(R, crosscorr, coefficients, N)
end function

```

```
function applyWienerFilter(input, coefficients, output, N)
    // y[n] = sum_{k=0}^{N-1} h[k] * x[n-k]
    for n from 0 to N-1 do
        sum <- 0
        for k from 0 to N-1 do
            if n - k >= 0 then
                sum <- sum + coefficients[k] * input[n - k]
            end if
        end for
        output[n] <- sum
    end for
end function

function computeMMSE(desired, output, N)
    // MMSE = (1/N) * sum (d[n] - y[n])^2
    sum <- 0
    for n from 0 to N-1 do
        e <- desired[n] - output[n]
        sum <- sum + e * e
    end for
    mmse <- sum / N
    return mmse
end function

function writeOutputToFile(fileName, outputSignal, N,
   mmseValue)
    open fileName for writing
    write "Filtered output: "
    for i from 0 to N-1 do
        write outputSignal[i] with one decimal and a space
    end for
    write newline
    write "MMSE: " and mmseValue with one decimal
    close file
end function
```

5 Results and discussions

In order to validate the implementation of the Wiener filter, we start with a short synthetic example in which the desired signal $d[n]$, the noisy observation $x[n]$ and the filtered output $y[n]$ all have length $N = 10$. Because the data record is very short and fully known, it is possible to check the numerical results in detail and to compare them with the theoretical solution of the Wiener–Hopf equations.

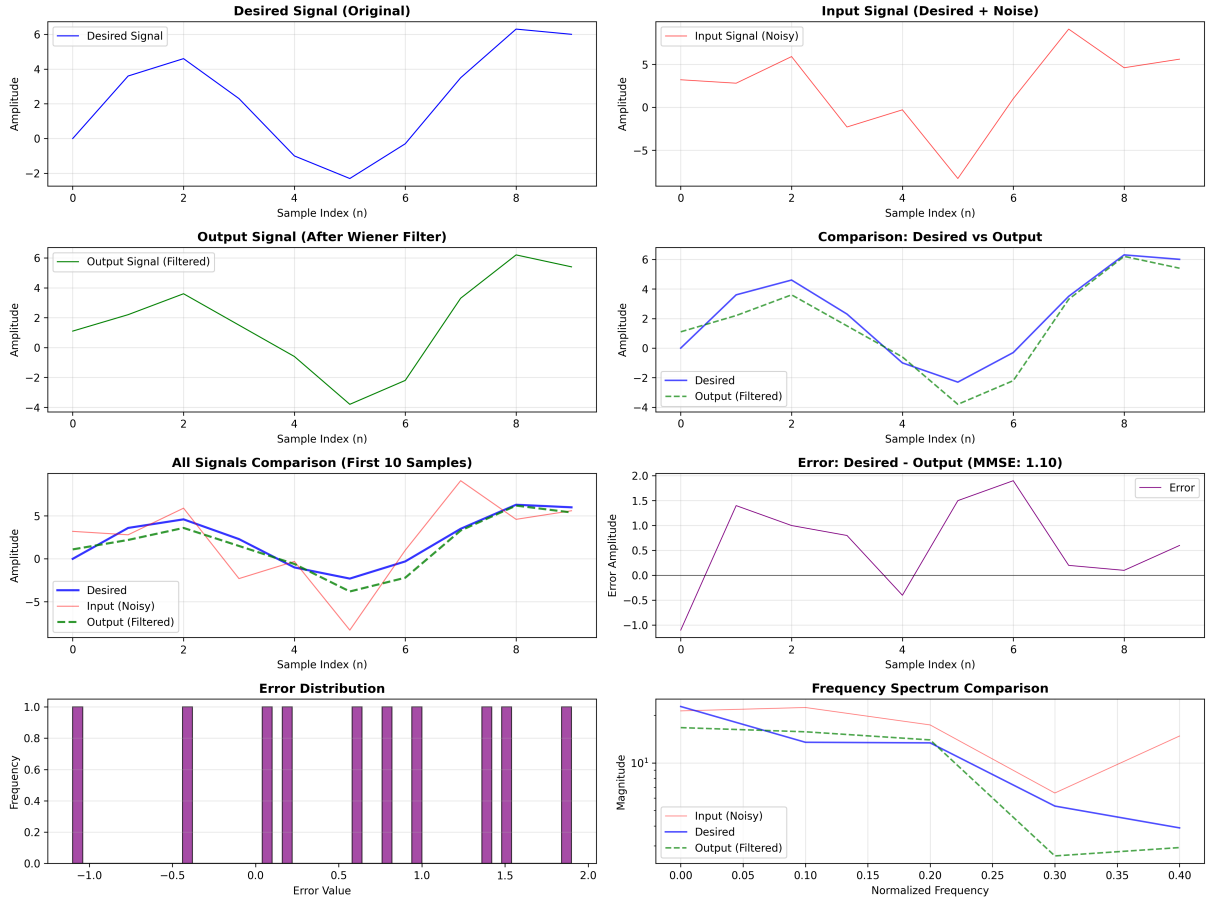


Figure 1: Analysis of the Wiener filter on the $N = 10$ sample test .

Figure 1 summarizes the behaviour of the filter on this data set. The top–left subplot shows the desired signal, which is a smooth sequence varying between approximately -2 and 6. This signal plays the role of the "clean" information that we would like to recover. The top–right subplot displays the corresponding input signal, generated by adding noise to the desired signal. Although the noise level is not extremely high, it clearly distorts the shape of the sequence and introduces several local fluctuations.

The second row of Figure 1 presents the output of the Wiener filter. The plot on the



left indicates that the filtered signal is much smoother than the input and follows the overall trend of the desired sequence. The subplot on the right directly compares $d[n]$ and $y[n]$. For almost all sample indices, the filtered output lies very close to the desired value; the largest visible deviations appear around the indices where the desired signal changes direction, which is expected for a finite–length linear filter.

A joint view of all three signals is given in the subplot *All Signals Comparison (First 10 Samples)*. In this zoomed–in representation, the desired signal forms a smooth curve, the noisy input oscillates around it, and the Wiener output tracks the desired signal while rejecting a significant portion of the noise. This confirms that the implemented filter behaves as an optimal linear estimator for this simple example.

The third row focuses on the error signal $e[n] = d[n] - y[n]$. The plot on the right shows that the errors remain small for all n, with no obvious outliers. The reported mean–square error is MMSE ≈ 1.10, which is consistent with the individual error samples. The histogram in the bottom–left subplot further illustrates that the errors are tightly concentrated within a narrow range around zero. Given that only ten samples are available, the distribution appears discrete, but it still shows that there is no large systematic bias in the estimation.

Finally, the bottom–right subplot compares the magnitude spectra of the three sequences. The noisy input has higher spectral magnitude over a wide frequency range, while the desired signal is dominated by low–frequency components due to its smooth variation. The spectrum of the Wiener–filtered output closely follows that of the desired signal in the low–frequency region and is significantly attenuated at higher frequencies. This frequency–domain behaviour confirms that the filter successfully suppresses the noise components that do not match the correlation structure of the desired signal.

Overall, this small test case demonstrates that the implemented Wiener filter recovers the main shape of the desired sequence from its noisy observation, achieves a low MMSE, and exhibits the expected low–pass–like behaviour in the frequency domain. This provides a solid sanity check for the correctness of the C++ and MIPS implementations before applying the same approach to longer and more realistic signals.

6 Conclusion

In this assignment, we have studied and implemented a Wiener filter for signal estimation in the presence of additive noise. Starting from the theoretical formulation of the minimum mean squared error criterion, we derived the Wiener–Hopf equations and

implemented the corresponding algorithm in a high-level language.

Using the verified high-level implementation as a reference, we then developed a MIPS assembly program that reads discrete-time signals from files, computes the correlation values, solves the linear system to find the optimal filter coefficients, and outputs both the filtered signal and the MMSE.

The project allowed us to practice not only digital signal processing concepts, but also low-level programming skills such as memory layout, floating-point operations, and structured decomposition of a complex algorithm into smaller procedures.

In future work, the implementation can be extended to support longer signals, different filter lengths, and more advanced numerical methods for solving linear systems. Furthermore, fixed-point arithmetic could be investigated to better approximate hardware-oriented designs.

References

[1] Alan V. Oppenheim and George C. Verghese. Chapter 11: Wiener filtering. In *Signals, Systems and Inference*. 2010. MIT OpenCourseWare, 6.011.

[2] Sophocles J. Orfanidis. *Optimum Signal Processing*. 2 edition, 2007. Online book.